Date : 16.03.2026

Learning CSS :

Selectors:

Type Selectors:

A **type selector** is sometimes called a *tag name selector* or *element selector* because it selects an HTML tag/element in your document. In the example below, we have used the span, em and strong selectors.

Example:

```
body {
        font-family: sans-serif;
}
span {
background-color: yellow;
}
```

Class Selectors:

The case-sensitive class selector starts with a dot (.) character. It will select everything in the document with that class applied to it. In the live example below, we have created a class called highlight and applied it to several places in the document. All elements with that class are highlighted.

Example:

```
.special{
        color:red;
}
```

Targeting classes on particular elements:

We do this by using the type selector for the element we want to target, with the class appended using a dot, with no white space in between.

Example:

```
span.highlight {
        background-color: yellow;
}
h1.highlight {
```

```
        background-color: pink;
}
```

ID Slectors:

The case-sensitive ID selector begins with a # rather than a dot character, but is used in the same way as a class selector. The difference is that an ID can be used only once per page, and elements can only have a single id value. An ID selector selects an element with a specific id, and you can precede the ID with a type selector to only target the element if both the element and ID match.

Example:

Html:

```
<h1 id="heading">ID selector</h1>
    <p>
      Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion
      daikon amaranth tatsoi tomatillo melon azuki bean garlic.
    </p>

    <p id="one">
      Gumbo beet greens corn soko <strong>endive</strong> gumbo gourd. Parsley
      shallot courgette tatsoi pea sprouts fava bean collard greens dandelion okra
      wakame tomato. Dandelion cucumber earthnut pea peanut soko zucchini.
    </p>
```

Css:

```
#one {
  background-color: yellow;
}

h1#heading {
  color: rebeccapurple;
}
```

The Universal Selector:

The universal selector, also known as a wildcard, selects everything - every single element in the document.

To use the universal selector, use the asterisk character, *.

Example:

```
* {
   property: value;
}
```

Combinators:

Child Combinator:

The **child combinator** (>) is placed between two CSS selectors. It matches only those elements matched by the second selector that are the direct children of elements matched by the first.

Elements matched by the second selector must be the immediate children of the elements matched by the first selector. This is stricter than the descendant combinator, which matches all elements matched by the second selector for which there exists an ancestor element matched by the first selector, regardless of the number of "hops" up the DOM.

Syntax:

```
/* The white space around the > combinator is optional but recommended. */
selector1 > selector2 {
        /* style properties */
}
```

Column combinator:

The **column combinator** (||) is placed between two CSS selectors. It matches only those elements matched by the second selector that belong to the column elements matched by the first.

Syntax;

```
/* The white space around the || combinator is optional but recommended. */
column-selector || cell-selector {
  /* style properties */
}
```

Next-sibling combinator:

The **next-sibling combinator** (+) separates two selectors and matches the second element only if it *immediately* follows the first element, and both are children of the same parent element.

Syntax:

/* The white space around the + combinator is optional but recommended. */

former_element + target_element { style properties }

Descendant combinator:

The **descendant combinator** — typically represented by a single space (" ") character — combines two selectors such that elements matched by the second selector are selected if they have an ancestor (parent, parent's parent, parent's parent's parent, etc.) element matching the first selector. Selectors that utilize a descendant combinator are called *descendant selectors*.

Syntax:

selector1 selector2 {

  /* property declarations */

}

Subsequent-sibling combinator:

The **subsequent-sibling combinator** (~, a tilde) separates two selectors and matches *all instances* of the second element that follow the first element (not necessarily immediately) and share the same parent element.

In the following example, the subsequent-sibling combinator (~) helps to select and style paragraphs that are both siblings of an image and appear after any image.

Syntax;

/* The white space around the ~ combinator is optional but recommended. */

former_element ~ target_element { style properties }

Pseudo classes :

A pseudo-class is a selector that selects elements that are in a specific state, for example, they are the first element of their type, or they are being hovered over by the mouse pointer. They tend to act as if you had applied a class to some part of your document, often helping you cut down on excess classes in your markup, and giving you more flexible, maintainable code.

Pseudo-classes are keywords that start with a colon. For example, :hover is a pseudo-class.

Example:

Html:

```
<a href="https://www.google.com">Google.com<a>
```

CSS:

```
a:link{
   color:blueviolet;
}
a:visited {
  color: rebeccapurple;
  font-weight: bold;
}

a:hover {
  color: hotpink;
}
```

User Action pseudo classes:

Some pseudo-classes only apply when the user interacts with the document in some way. These **user-action** pseudo-classes, sometimes referred to as **dynamic pseudo-classes**, act as if a class had been added to the element when the user interacts with it.

- :hover — this only applies if the user moves their pointer over an element, typically a link.
- :focus — only applies if the user focuses the element by clicking or using keyboard controls.

Pseudo-class uses single colon:

:hover

Most Commonly used Pseudo classes:

:hover → Applies styles when the user places the mouse pointer over an element.

:focus → Applies styles when an element (like an input) is selected or clicked into.

:active → Applies styles while an element is being clicked.

:first-child → Selects an element that is the first child of its parent.

:last-child → Selects an element that is the last child of its parent.

:nth-child(n) → Selects an element based on its position (number) among its siblings.

:not(selector) → Selects elements that do NOT match the specified selector.

:checked → Selects radio buttons or checkboxes that are selected.

:disabled → Selects form elements that are disabled.

:valid → Selects form inputs with valid data according to validation rules.

:invalid → Selects form inputs with invalid data according to validation rules.

Pseudo Elements :

Pseudo-elements behave in a similar way. However, they act as if you had added a whole new HTML element into the markup, rather than applying a class to existing elements.

Pseudo-elements uses double colon:

:before

Example:

Html:

```
<article>
    <p>
      Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette
      tatsoi pea sprouts fava bean collard greens dandelion okra wakame tomato.
      Dandelion cucumber earthnut pea peanut soko zucchini.
    </p>
  </article>
```

CSS:

```
article p::first-line {
  font-size: 120%;
  font-weight: bold;
}
```

Combining Pseudo classes and Pseudo Element:

If you wanted to make the first line of the first paragraph bold you could chain the :first-child and ::first-line selectors together.

```
article p:first-child::first-line {
```

```
  font-size: 120%;
  font-weight: bold;
}
```

Generating Content with ::before and ::after

There are a couple of special pseudo-elements, which are used along with the content property to insert content into your document using CSS. This technique is called **generated content**.

Example:

Html:

```
<p class="box1">Content in the box in my HTML page.</p>
```

Css:

```
.box1::before{
    content: "This should show Before the other content. ";
    background-color: yellow;
}
.box1::after {
  content: " ➡";
}
.box1::before {
  content: "";
  display: block;
  width: 100px;
  height: 100px;
  background-color: rebeccapurple;
  border: 1px solid black;
}
```

CSS Attribute Selectors

Many HTML elements have attributes.

HTML attributes:

- Provide additional information about HTML elements.

- Are always specified in the start (or opening) tag.
- Usually come in name/value pairs such as name="value".
- The value in a name/value pair should be enclosed in quotation marks.
- One of the most popular HTML attributes you may have come across is the `href` attribute, which is added to the opening `<a>` tag and specifies the URL the `<a>` tag links to:
- `<a href="https://www.freecodecamp.org/">The best place to learn to code for free!</a>`
- `The attribute selector matches and selects HTML elements based on the presence of an attribute or a specific attribute value.`

1)The [attribute] **Selector**

To use the attribute selector, use a pair of square brackets, [], to select the attribute you want.

The general syntax for attribute selectors is the following:

element[attribute]

elements that have the attribute attr present are selected, regardless of the specific value of attr:

```
a[attr] {
   property: value;
}
```

2)The [attribute="value"] **Selector**

You can specify the value of the attribute using the following syntax:

element[attribute="value"]

So, if you want to style a elements with an attr attribute that has an **exact** value of 1, you would do the following:

```
a[attr="1"] {
   property: value;
}
```

3)The [attribute^="value"] **Selector**

You could also specify that the value of the attribute **starts** with a specific character using the following syntax:

element[attribute^="value"]

For example, if you wanted to select and style any a elements that have an attr attribute with a value that starts with www, you would do the following:

```
a[attr^="www"] {
   property: value;
}
```

4)The [attribute$="value"] **Selector**

You could also specify that the value of the attribute **ends** with a specific character using the following syntax:

```
element[attribute$="value"]
```

For example, if you wanted to select a elements that have an attr attribute name with a value that ends with .com, you would do the following:

```
a[attr$=".com"] {
   property: value;
}
```

5)The [attribute*="value"] **Selector**

You can also specify that the attribute value contains a specific substring - this selector is known as the Attribute Contains Substring Selector and has the following syntax:

```
element[attribute*="value"]
```

```
a[attr*="free"] {
   property:value;
```

} The code above selects a elements with an attr attribute name when the string free is present in the attribute's value - even as a substring (a substring is a word inside another word).

6)The [attribute~="value"] **Selector**

You can also specify that the selector matches an attribute value that contains a whole word using the following syntax:

```
element[attribute~= "value"]
```

For example, if you wanted to select a elements that have an attr attribute name with a value that contains the word free, you would do the following:

```
a[attr~= "free"] {
   property: value;
}
```

Grouping CSS Selector

With the grouping selector, you can target and style more than one element at once.

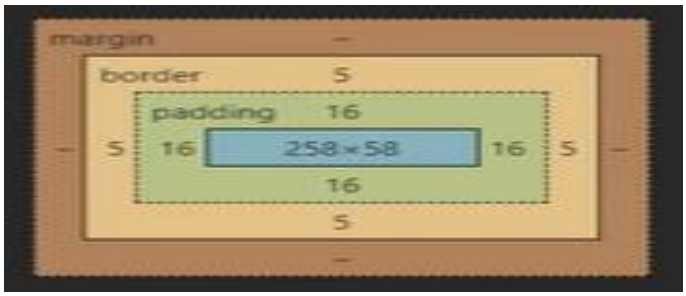
```
div, p, span {
   property: value;
}
```

The Box Model

The box model consist of contents, padding, margin and border.

The default margin size is 8px.

The default display is block.



I can put box sizing in that we have two

1) content-box (default)
2) border-box

content-box (Default):

Calculation: Total Width = width + padding + border.

Behavior: Setting a 100px width with 10px padding makes the box 120px wide.

Use Case: When you want the content area to be a specific size regardless of padding/border.

border-box:

Calculation: Total Width = width (includes padding and border).

Behavior: Setting a 100px width with 10px padding keeps the total box at 100px wide, shrinking the content area instead.

Use Case: Preferred for most modern layouts to avoid layout breakage.

Types of Display Properties

1) Block – Takes full width
2) Inline – Stays in line
3) Flex – Flexible box
4) Grid – Two dimensional layout
5) None – Disappears

These properties controls how an element appears and behaves in the layout

Block-level elements

- div
- p
- h1 to h6
- header
- footer

These elements takes up the full width and starts on the new line.

Inline Elements

- span
- button
- input
- img
- a

These elements sits next to other elements on the same line.

Block and Inline boxes

In CSS we have several types of boxes that generally fit into the categories of **block boxes** and **inline boxes**.

Boxes have an **inner display type** and an **outer display type**.

In general, you can set various values for the display type using the display property.

If a box has a display value of block, then:

- The box will break onto a new line.
- The width and height properties are respected.
- Padding, margin and border will cause other elements to be pushed away from the box.

- If width is not specified, the box will extend in the inline direction to fill the space available in its container. In most cases, the box will become as wide as its container, filling up 100% of the space available.
- Some HTML elements, such as <h1> and <p>, use block as their outer display type by default.

If a box has a display type of inline, then:

- The box will not break onto a new line.
- The width, height, and top and bottom margins will have no effect.
- **Top and bottom** padding and borders will change the size of the box without affecting the position of surrounding content, which can cause overlapping.
- **Left and right** padding, margins, and borders will affect the position of surrounding inline content.
- Some HTML elements, such as <a>, , and use inline as their outer display type by default.

Flexbox:

The **CSS flexible box layout** module defines a CSS box model optimized for user interface design, and the layout of items in one dimension. In the flex layout model, the children of a flex container can be laid out in any direction, and can "flex" their sizes, either growing to fill unused space or shrinking to avoid overflowing the parent. Both horizontal and vertical alignment of the children can be easily manipulated.

Inner and outer display type:

block and **inline** display values are said to be **outer display** types — they affect how the box is laid out in relation to other boxes around it. Boxes also have an **inner display** type, which dictates how elements inside that box are laid out.

CSS display: inline-block – Simple Small Notes

Definition:
inline-block is a display value that behaves **like inline and block together**.

Key Points

- **Does not start on a new line** (like inline).
- **Width and height can be set** (like block).
- **Padding, margin, and border work properly**.
- **Elements stay in the same line**.

- Size depends on **content unless width/height is given**.

Example

```
.box {
  display: inline-block;
  width: 100px;
  height: 50px;
  margin: 10px;
}
```

Output Behavior

[BOX] [BOX] [BOX]

Handling Conflicts in CSS

The word **"Cascading"** means **styles flow and override each other based on certain rules**.

If multiple styles are written for the same element, CSS decides **which one should be applied**.

CSS stands for **Cascading Style Sheets**, where the word **cascading** means that multiple CSS rules can apply to the same element and CSS decides which rule should be used. Sometimes while working on a project, a style you wrote may not work as expected because another CSS rule is overriding it. This usually happens when two rules try to apply different values to the same property of the same element. CSS resolves these conflicts using mechanisms like the **cascade** and **specificity**, which determine which rule has higher priority and should be applied. Another important concept is **inheritance**, where some CSS properties automatically take values from the parent element, while others do not. Because of cascade, specificity, and inheritance, the style applied to an element may not always be the one you initially expect, but understanding these concepts becomes easier with practice.

Cascade:

Stylesheets cascade. At a very simple level, this means that the origin and the order of CSS rules matter. When two rules both have equal specificity, the one that is defined last in the stylesheet is the one that will be used.

Example:

```
h1 { color: red; }

 h1 { color: blue; }
```

we have two rules that could apply to the <h1> element. The <h1> content ends up being colored blue. This is because both the rules are from the same source, have an identical element selector, and therefore, carry the same specificity, but the last one in the source order wins.

Specificity:

Specificity is an algorithm that the browser uses to decide which property value is applied to an element. If multiple rules have different selectors that set different values for the same property and target the same element, specificity decides the property value that gets applied to the element.

Type (Element) Selector

- Has **low specificity**.
- Selects **all elements of that type** on the page.
- Has **less weight**.
- **Pseudo-element selectors** have the **same specificity** as element selectors.

Class Selector

- Has **higher specificity than element selectors**.
- Selects elements with a **specific class attribute**.
- Has **more weight** than element selectors.
- **Attribute selectors** and **pseudo-classes** have the **same specificity as class selectors**.

ID Selector

- Has **higher specificity than class selectors**.
- Selects **one unique element** with a specific **id value**.
- Has the **highest weight among these selectors**.

Inheritance:

Inheritance also needs to be understood in this context — some CSS property values set on parent elements are inherited by their child elements, and some aren't.

For example, if you set a color and font-family on an element, every element inside it will also be styled with that color and font, unless you've applied different color and font values directly to them.

CSS Universal Property Values (Inheritance Control)

CSS has **5 special values** that control how properties behave.

1. inherit

- The element **takes the value from its parent**.

- It **forces inheritance**.

Example:

```
p {
  color: inherit;
}
```

If parent color is **blue**, the <p> will also become **blue**.

2. initial

- Sets the property to its **original default value defined by CSS**.

Example:

```
p {
  color: initial;
}
```

The text will go back to the **default browser color (usually black)**.

3. revert

- Resets the property to the **browser's default styling**.
- Removes styles you added.

Example:

```
p {
  color: revert;
}
```

It will return to the **browser's default style**.

4. revert-layer

- Resets the value to the **style from a previous CSS layer**.
- Mostly used in **advanced CSS cascade layers**.

Simple idea:
It **goes back to the previous layer's value**.

5. unset

- Resets property to its **natural behavior**.

Meaning:

- If the property **normally inherits** → acts like inherit
- If the property **does not inherit** → acts like initial

Example:

```
p {
  color: unset;
}
```

Since **color normally inherits**, it will **take the parent's color**.

inherit → parent value

initial → CSS default

revert → browser default

revert-layer → previous layer value

unset → natural behavior

Understanding the Cascade:

- **Inheritance** explains why elements inside HTML (like a paragraph inside many tags) get styles from their **parent element**, for example a paragraph getting the **color from the body**.
- We can **change styles anywhere in the document** by:
    - Applying CSS directly to an **element selector**
    - Using a **class**
- Sometimes **multiple CSS rules apply to the same element with different values** for the same property.
- The **CSS Cascade** decides **which rule will finally apply**.
- The browser checks **three factors** to determine the final style.
- These factors are checked in **increasing order of importance** (later ones override earlier ones).

Three Factors of the Cascade

1. **Source Order**
    - The rule written **later in the CSS file** usually overrides earlier rules.
2. **Specificity**

    - Selectors with **higher priority** (like ID over class over element) override others.

3. **Importance**
    - Rules with **!important** have the **highest priority**.

- Using these three rules, the **browser decides which CSS style should be applied to an element**.

Effect of CSS Location (Where CSS is Written)

The priority of a CSS rule also depends on where the stylesheet comes from.
Different styles can come from three sources:

1. User Agent Stylesheet
2. User Stylesheet
3. Author Stylesheet

The browser follows a specific order to decide which style overrides others.

1) User Agent Style sheet
   This is the default CSS provided by the browser.

   Example:
   Browsers automatically style elements like:

   - headings (h1, h2)
   - paragraphs
   - links
2) User Stylesheet
   A user stylesheet is custom CSS set by the user in their browser.
3) Author Stylesheet
   This is the CSS written by the web Developers
   This is the CSS we normally write in:
   - external CSS files
   - <style> tags
   - inline styles

Order of Overriding CSS Declarations

When **multiple styles conflict**, the browser follows this order
(from **lowest priority to highest priority**).

Later rules **override earlier ones**.

1. **User Agent Styles (Lowest Priority)**

Browser default styles.

Example:

```
h1 {
  font-size: 32px;
}
```

2. **Normal User Styles**

Styles set by the **user** in their browser.

Example:

```
body {
  font-size: 20px;
}
```

3. **Normal Author Styles**

CSS written by **developers**.

Example:

```
p {
  font-size: 16px;
}
```

These usually override browser defaults.

4. **Important Author Styles**

Developer CSS using **!important**.

Example:

```
p {
  font-size: 16px !important;
}
```

This overrides **normal user styles**.

5. **Important User Styles**

If the user writes **!important** in their stylesheet, it overrides developer styles.

Example:

```
p {
  font-size: 30px !important;
}
```

User accessibility settings **take priority over developer styles**.

6. **Important User Agent Styles (Highest Priority)**

Rare case where browser default styles use **!important**.

These override everything.

Values and Units

CSS Value:

CSS values define what types of value are valid for each CSS property. For example, you can specify colors for the values of color or border-color, but not lengths or percentages.

Numbers, Lengths and Percentages

Data type	Description
`<integer>`	An `<integer>` is a whole number such as `1024` or `-55`.
`<number>`	A `<number>` represents a decimal number — it may or may not have a decimal point with a fractional component. For example, `0.255`, `128`, or `-1.2`.
`<dimension>`	A `<dimension>` is a `<number>` with a unit attached to it. For example, `45deg`, `5s`, or `10px`. `<dimension>` is an umbrella category that includes the `<length>`, `<angle>`, `<time>`, and `<resolution>` types.
`<percentage>`	A `<percentage>` represents a fraction of some other value. For example, `50%`. Percentage values are always relative to another quantity. For example, an element's length is relative to its parent element's length.

Lengths

The numeric type you will come across most frequently is <length>. For example, 10px (pixels) or 30em. There are two types of lengths used in CSS — relative and absolute. It's important to know the difference in order to understand how big things will become.

Absolute length units

The following are all **absolute** length units — they are not relative to anything else, and are generally considered to always be the same size.

Unit	Name	Equivalent to
`cm`	Centimeters	1cm = 37.8px = 25.2/64in
`mm`	Millimeters	1mm = 1/10th of 1cm
`Q`	Quarter-millimeters	1Q = 1/40th of 1cm
`in`	Inches	1in = 2.54cm = 96px
`pc`	Picas	1pc = 1/6th of 1in
`pt`	Points	1pt = 1/72nd of 1in
`px`	Pixels	1px = 1/96th of 1in

Most of these units are more useful when used for print, rather than screen output. For example, we don't typically use cm (centimeters) on screen. The only value that you will commonly use is px (pixels).